Linux: appels système (I2181-B)

SOLUTION

1. Exercice sur les appels systèmes : read – write

Lorsqu'on introduit une phrase trop longue, ici la phrase « cette phrase dépasse les 10 caractères du buffer! » :

```
alegrand@courslinux:LAS_01_solutions$ ./ex1.1
Introduisez votre ligne (max 10 caractères, \n inclus):
cette phrase dépasse les 10 caractères du buffer!
cette phra
Introduisez votre ligne (max 10 caractères):
se dépass
Introduisez votre ligne (max 10 caractères):
e les 10 c
Introduisez votre ligne (max 10 caractères):
aractères
Introduisez votre ligne (max 10 caractères):
 du buffer
Introduisez votre ligne (max 10 caractères):
!

Introduisez votre ligne (max 10 caractères):
```

Le programme traite cette phrase en plusieurs tours de boucle. A chaque itération, 10 caractères sont lus à l'entrée standard (càd dans le buffer clavier), jusqu'à ce que le caractère \n soit lu. Le programme vous demande alors d'entrer une nouvelle phrase.

Traitement 1 :

Si le programme se termine (*exit(0)*) lorsqu'une ligne trop longue est entrée au clavier, ici la phrase « cette phrase dépasse les 10 caractères du buffer! » :

```
alegrand@courslinux:LAS_01_solutions$ ./ex1.1
Introduisez votre ligne (max 10 caractères, \n inclus):
cette phrase dépasse les 10 caractères du buffer!
Erreur: la ligne introduite fait plus de 10 caractères.
alegrand@courslinux:LAS_01_solutions$ se dépasse les 10 caractères du buffer!
-bash: se: command not found
```

On constate que, le buffer du clavier n'ayant pas été vidé avant que le programme se termine, le contenu restant s'affiche directement dans l'invite de commande ! Comme \n est présent parmi ces caractères, la ligne est exécutée par le terminal. Cette ligne provoque évidemment une erreur car bash ne comprend pas la commande.

Notez que cela constitue un risque de sécurité. En effet, comme le montre l'exemple suivant, si ce qui n'a pas été lu dans le buffer clavier correspond à une commande, elle sera exécutée !

```
alegrand@courslinux:LAS_01_solutions$ ./ex1.1
Introduisez votre ligne (max 10 caractères, \n inclus):
1234567890 ls -l
Erreur: la ligne introduite fait plus de 10 caractères.
alegrand@courslinux:LAS_01_solutions$  ls -l
total 172
-rwxr-xr-x 1 alegrand alegrand 28904 Feb 25 10:17 ex1.1
-rw-r--r-- 1 alegrand alegrand  2367 Feb 25 10:17 ex1.1.c
-rw-r--r-- 1 alegrand alegrand  2392 Feb 25 10:17 ex1.1.o
-rwxr-xr-x 1 alegrand alegrand 28960 Feb 24 16:25 ex1.2
-rw-r--r-- 1 alegrand alegrand  2231 Feb 23 12:33 ex1.2.c
-rw-r--r-- 1 alegrand alegrand  2976 Feb 24 16:25 ex1.2.o
-rwxr-xr-x 1 alegrand alegrand 29136 Feb 24 16:25 ex1.3
-rw-r--r-- 1 alegrand alegrand  2239 Feb 23 21:48 ex1.3.c
-rw-r--r-- 1 alegrand alegrand  3888 Feb 24 16:25 ex1.3.o
-rwxr-xr-x 1 alegrand alegrand 16568 Feb 24 16:25 ex1.4
-rw-r--r-- 1 alegrand alegrand  2514 Feb 23 22:50 ex1.4.c
-rw-r--r-- 1 alegrand alegrand  1378 Feb 27  2024 lievreETtortue.txt
-rw-r--r-- 1 alegrand alegrand   670 Feb 23 22:42 Makefile
-rw-r--r-- 1 alegrand alegrand 19032 Feb 24 13:57 utils_v1.o
alegrand@courslinux:LAS_01_solutions$
```

La commande « ls -l\n » qui restait dans le buffer clavier à la sortie du programme est exécutée par bash. Cela constitue clairement un risque de sécurité ! Par conséquent, il faut systématiquement vider le buffer système du clavier avant de quitter le programme, comme cela est demandé dans le traitement 2 de cet exercice.

3. Exercice avec redirection de l'entrée standard

Reprenez l'exercice 2 et testez-le en redirigeant l'entrée standard :

```
lireEtEcrire Maj min < lievreETtortue.txt
```

Le résultat est-il celui qui est attendu ? Pourquoi ?

NON.

Dans un terminal, `read` lit dans le buffer système du clavier dès que la touche ENTER est enfoncée. Le terminal/clavier est un « character device », comme le montre la première lettre affichée par la commande ls -l : 'c'.

```
alegrand@courslinux:LAS_01_solutions$ ls -l /dev/tty
crw-rw-rw- 1 root tty 5, 0 Feb 27 10:54 /dev/tty
```

Dans un *character device*, les caractères '\n' sont traités comme des passages à la ligne. Le dernier caractère du buffer clavier sera donc toujours '\n'.

Dans le cas où un fichier est redirigé vers l'entrée standard, la lecture se fait dans un fichier. Or, un fichier se trouve sur un disque, qui est un « block device », comme le montre la première lettre affichée par la commande ls -l : 'b'.

```
alegrand@courslinux:LAS_01_solutions$ ls -l /dev/sda
brw-rw---- 1 root disk 8, 0 Oct 18 13:53 /dev/sda
```

Dans un *block device*, le caractère de fin de ligne '\n' est alors considéré comme n'importe quel caractère. `read` considère donc le fichier comme une longue chaîne et n'interrompra la lecture que si la fin du fichier est atteinte ou si le nombre maximal de caractères demandé a été lu.

Exemple 1 : le vidage de buffer se fait caractère par caractère, jusqu'à la lecture d'un '\n'

```
$ ./ex1.2 Maj min < lievreETtortue.txt
$ cat min
et leur fait arpenter les landes.
Ayant, dis-je, du temps de reste pour brouter,
$ cat Maj
Moi l'emporter ! et que serait-ce
si vous portiez une maison ?
```

Les 2 lignes ci-dessus sont écrites dans min car elles font exactement 81 caractères (avec le dernier caractère = '\n').
Les 2 dernières lignes du fichier sont copiées dans Maj car elles font moins de 81 caractères (la fin de fichier est atteinte avant de lire 80 caractères).
Ces lignes sont donc les seules qui ne sont pas ignorées par le programme.

Exemple 2 : le vidage de buffer se fait avec le buffer de 80+1 caractères, jusqu'à ce que le dernier caractère lu soit un '\n'

```
$ ./ex1.2 Maj min < lievreETtortue.txt
$ cat min
$ cat Maj
```

Aucune ligne n'est écrite dans les fichiers min et Maj car, dès la 1e lecture de 81 caractères, le programme détecte une ligne trop longue et vide le buffer. Malheureusement, ce vidage de buffer ne s'arrêtera qu'à la fin du fichier. En effet, le programme va lire l'entièreté du fichier sans jamais rencontrer \n comme 81e caractère lu.

Comment faut-il modifier le programme pour qu'il produise le même effet, qu'il y ait redirection ou pas ?

Pour traiter les lignes une par une, il faudrait traiter le buffer de lecture caractère par caractère, que ce soit pour la lecture ou le vidage de buffer dans le cas d'une ligne trop longue. Dans ce cas, le programme détecte si le caractère qui a été lu est une fin de ligne '\n' et le traite en conséquence.

→ cf. **ex1.3.v1.c**

Cette lecture caractère par caractère étant peu efficace, nous vous montrons une seconde solution qui lit 81 caractères à la fois (lecture avec buffer). Elle est clairement plus complexe que la première solution et sort du cadre de ce cours.

→ cf. **ex1.3.v2.c**

4. Bonus - Comparaison entre fwrite et write

Le programme utilisant les fonctions de la librairie C est (beaucoup) plus rapide que celui utilisant les appels système.

```
alegrand@courslinux:LAS_01_solutions$ time ./ex1.4 fwrite
Test fwrite ... cela est rapide car fwrite bufferise les I/O

real    0m1.196s
user    0m0.720s
sys     0m0.285s
alegrand@courslinux:LAS_01_solutions$ time ./ex1.4 write
Test write ... cela va prendre du temps

real    1m7.379s
user    0m7.796s
sys     0m59.488s
```

Cela s'explique pour deux raisons: les accès disque sont lents et les fonctions IO des librairies C sont bufferisées.

1) Version utilisant *fwrite*

La fonction *fwrite* n'écrit sur le disque que lorsque son buffer est plein, comme le montre cette ligne produite par le script *lastrace.sh*:

```
[PID=49286] write(3, "\0x\230\0\1x\230\0\2x\230\0\3x\230\0\4x\230\0\5x\230\0\6x\230\0"..., 4096) = 4096
```

On constate que le programme utilisant *fwrite* écrit 4.096 bytes à la fois sur le disque (i.e. lorsque le buffer est plein). La commande suivante vous indique que la première version du programme réalise 9766 appels système *write*.

```
lastrace.sh ex01.4.v1 | grep write | wc -l
```

2) Version utilisant *write*

La seconde version du programme effectue un appel système *write* pour chacun des 10.000.000 d'entiers à écrire dans le fichier car *write* n'est pas bufferisé. La commande suivante affiche bien que ce second programme réalise 10.000.000 d'appels à write:

```
lastrace.sh ex01.4.v2 | grep write | wc -l
```

Le programme écrit seulement 4 bytes à chaque appel de write, comme l'indique le script lastrace.sh:

```
[PID=52997] write(3, "\342\3\0\0", 4) = 4
```

➔ En conclusion, le programme utilisant les syscalls est plus lent parce qu'il effectue 1000 fois plus d'accès disque que le programme utilisant les fonctions standard C qui, elles, sont bufferisées.